SECURE VISITOR COUNTER WITH CROWD CONTROL

Submitted as a partial fulfillment of
the requirement for the Mini Project 1B Course of
Semester 4, AY 2024-2025

Submitted by

Samiksha Gaonkar	Roll No: 14
Pritika Mediboina	Roll No: 26
Yukita Jaiswal	Roll No: 19
Jancy Patlikadan	Roll No: 68

Supervised by
Freda Venessa Carvalho
Jayarajesh Vattam

Department of Electronics & Telecommunication Engineering

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai-400 070

University Of Mumbai

2024-25

The Bombay Salesian Society's
Don Bosco Institute of Technology
(Affiliated to the University of Mumbai)
Premier Automobiles Road, Kurla, Mumbai - 400070

Certificate

This is to certify that the Mini project entitled **Secure Counter with Crowd Control** is a work of

Samiksha Gaonkar	Roll No: 14
Pritika Mediboina	Roll No: 26
Yukita Jaiswal	Roll No: 19
Jancy Patlikadan	Roll No: 68

submitted as partial fulfillment of the requirement for the Mini Project 1B of **"Semester 4"** in **"Second Year of Engineering AY 2024-2025"**.

(**Freda Carvalho**)
(**Asst. Prof. Dept of EXTC**)

(**Jayarajesh Vattam**)
(**Asst. Prof.Dept of EXTC**)

(**Madhavi Pednekar**)
(**HOD, Dept. EXTC**)

The Bombay Salesian Society's

Don Bosco Institute of Technology

(Affiliated to the University of Mumbai)
Premier Automobiles Road, Kurla, Mumbai - 400070

Mini Project 1B Report Approval

This Mini project report entitled **'Secure Visitor Counter with Crowd Control'** by **Samiksha Gaonkar, Pritika Mediboina, Yukita Jaiswal, Jancy Patlikadan** is approved for the completion of Mini Project 1B course **of Sem 4 of AY 2024-2025 in Dept. of Electronics & Telecommunication Engineering.**

Examiners

1. _____

2. _____

Date : / /

Place : **Kurla, Mumbai**

Contents

Certificate

1 Introduction **1**

1.1 Project Motivation 2

1.2 Project Objective & Outcomes 3

1.2.1 Objective 3

1.2.2 Outcomes 3

1.2.3 Applications 4

2 Methodology **5**

3 Project Implementation **7**

3.1 Block diagram 7

3.2 Circuit Diagram 8

3.3 List of Components and Budget 10

3.4 Results 11

3.5 Special Features 13

4 Conclusion **14**

Bibliography **14**

A Datasheet-1 **16**

B Datasheet-2 **22**

C Datasheet-3 **25**

D Datasheet-4 **29**

E Datasheet-5 **32**

5 Codes **34**

List of Figures

3.1 Block Diagram 8
3.2 Circuit Diagram 8
3.3 Model Display 11
3.4 Circuit Assembly 11

List of Tables

3.1 List of components and their cost 10
3.2 Visitor Count Results with Corresponding System Displays 12
3.3 Special Features Operation Displays 13

Abstract

This project presents a Visitor Counter and Crowd Control System that monitors and controls the number of people entering a specific area using basic sensor-based counting. Infrared (IR) sensor placed at entry and exit points detect movement and maintain a real-time count of visitors.

The system updates the visitor count as individuals enter and exit. Once the number of people inside reaches a predefined maximum limit, further entry is restricted automatically. A display system shows a "Maximum Limit Reached" warning and entry is disabled until the count decreases.

In special cases, access can still be granted using a password-based identity verification system. Authorized personnel can enter a secure password via a keypad to override the restriction and gain access even when the limit is reached.

This system is suitable for managing entry in controlled environments like halls, libraries, exam centers, or any area where crowd regulation is necessary. It provides a simple, effective, and low-cost solution for basic crowd control and visitor tracking.

Chapter 1

Introduction

In today's world, managing and monitoring crowd flow in public and private spaces is crucial for ensuring safety, security, and efficient space utilization. A Secure Visitor Counter with Crowd Control System offers a smart solution to address this need. This system is designed to accurately track the number of people entering and exiting a designated area, such as a hall or room, while incorporating advanced security features to prevent unauthorized access.

The system includes a password identification mechanism that ensures only authorized personnel can reset or modify its settings. Additionally, it enforces a maximum occupancy limit to prevent overcrowding. Once the predefined limit is reached, the system can trigger alerts or automatically restrict further entry, thereby enhancing crowd control and safety compliance.

Key features of the system include real-time visitor counting, password-protected access for system settings, alerts for exceeding maximum occupancy, a display showing the current number of people present, and optional integration with gates or doors for automated access control.

This secure and intelligent system is ideal for applications in malls, auditoriums, conference halls, offices, and other environments where crowd control and safety are critical priorities. [1]

1.1 Project Motivation

In an increasingly connected world, public safety and efficient space utilization are top priorities—especially in venues like malls, airports, stadiums, religious places, hospitals, and government buildings. The growing number of people frequenting these places brings with it the challenge of managing crowds effectively and ensuring that every individual is accounted for without compromising security or operational efficiency.

Traditional methods—such as security personnel manually tracking entries and exits, or using simple mechanical turnstiles—are no longer sufficient. They can be bypassed, miscounted, or overwhelmed during emergencies. Real-world incidents underline the urgent need for intelligent crowd control and secure visitor monitoring systems.

Case Studies and Incidents:
The 2022 Itaewon Crowd Crush (South Korea): A tragic example of poor crowd management, where over 150 people lost their lives during Halloween celebrations in Seoul. There were no real-time crowd monitoring systems in place to detect and respond to the dangerously growing density.

COVID-19 Pandemic (2020-2022): During the global health crisis, many institutions struggled to monitor and limit the number of visitors in real time. This delay in implementing occupancy limits led to virus spread in places like supermarkets and public transport systems. A real-time counter could have automatically restricted entries when capacity was reached.

Stampede at Kumbh Mela (India, various years): With millions attending religious gatherings, stampedes have repeatedly occurred due to overcrowding and lack of crowd flow regulation. Intelligent counters integrated with crowd control could mitigate such disasters. This project is motivated by the need for a reliable and automated system that can accurately monitor the number of people within a space and take appropriate action when occupancy limits are reached. With growing awareness around health protocols, emergency preparedness, and the importance of regulated access, developing a secure and intelligent visitor counter is both timely and impactful.

1.2 Project Objective & Outcomes

1.2.1 Objective

The primary objectives of the Secure Visitor Counter with Crowd Control system using IR sensor are as follows:

- To develop an automated system that accurately counts the number of visitors entering and exiting a space.
- To monitor real-time occupancy and compare it against a predefined limit.
- To trigger alerts or restrict entry when the crowd exceeds the allowed capacity.
- To enhance security by optionally integrating authentication methods like RFID or facial recognition.
- To create a cost-effective and scalable solution suitable for various environments.
- To reduce dependency on manual supervision for crowd control and visitor tracking.

1.2.2 Outcomes

The expected outcomes of this project include:

- A working prototype that automatically counts people in real time using sensors.
- A microcontroller-based system capable of processing entry/exit data and managing occupancy limits.
- Visual/auditory alert system activated when maximum capacity is reached.
- Optional integration of access control mechanisms (e.g., RFID, face recognition).

- Improved safety, security, and operational efficiency in managing public or private spaces.
- A user-friendly, low-maintenance system ready for real-world deployment.[2]

1.2.3 Applications

list of applications for your Secure Visitor Counter with Crowd Control project:

- Classrooms and Seminar Halls – To control access during lectures or events and prevent overcapacity.
- Libraries and Study Halls – For tracking the number of students and managing crowd levels during peak hours.
- Offices and Workspaces – To monitor and control the number of employees or visitors inside meeting rooms or restricted areas.
- Event Venues – For crowd control and secure entry management at exhibitions, conferences, or concerts.
- Hospitals and Clinics – To manage waiting room capacity, ensuring social distancing and safety compliance.

Chapter 2

Methodology

The Secure Visitor Counter with Crowd Control system is designed to monitor and manage the number of people entering and exiting a room using two IR sensors. These sensors detect movement at the entry and exit points, and the data is processed by an Arduino Uno to maintain an accurate visitor count. A servo motor is used to automate the door mechanism, allowing entry only if the number of visitors is below a predefined limit. The current visitor count is continuously displayed on a 16x2 LCD screen. A 4x4 matrix keypad is integrated to provide administrative control, enabling functions like resetting the count or modifying the maximum limit. If the room reaches its maximum capacity, the system automatically blocks further entry until someone exits, ensuring safe and efficient crowd management.[3]

- Component-wise Explanation
- 1. Arduino Uno (Center of the Circuit) The Arduino Uno serves as the main controller of the project.

  It connects to all input/output devices including the keypad, IR sensors, servo motor, and LCD.

  It's powered either through a USB or external DC adapter.
- 2. IR Sensors (Bottom Section, 2 Black Modules) These two sensors detect motion for entry and exit.

Each sensor has three connections: VCC (5V), GND, and OUT (signal pin).

The signal wires from the IR sensors are connected to digital pins on the Arduino (likely D2 and D3 in the image).

- 3. 16x2 LCD Display (Right Center) This display shows the visitor count, status messages, or access prompts.

  It is likely connected via an I2C module, simplifying the wiring (only 4 wires: VCC, GND, SDA, and SCL).

  SDA and SCL are connected to A4 and A5 on Arduino respectively.

- 4. 4x4 Matrix Keypad (Top Right Corner) The keypad is used for password protection or manual inputs (like resetting, setting limit, admin control).

  It connects to the Arduino using 8 digital pins, typically labeled R1–R4 (rows) and C1–C4 (columns).

- 5. Servo Motor (Top Center) The servo motor simulates the automatic door opening/closing mechanism. The signal pin is likely connected to a PWM-capable digital pin (like D9 or D10).

Chapter 3

Project Implementation

The Visitor Counter System with Crowd Control is an automated solution developed using an Arduino Uno, designed to count the number of visitors entering and exiting a space. The system uses infrared (IR) sensors to detect movement, a servo motor to control the door mechanism, and an LCD display to show the current count of people inside. When the number of visitors reaches a predefined limit, the system prevents further entry to ensure safety and manage crowd control. This project is ideal for locations where space capacity needs to be regulated such as classrooms, shops, or offices.[4]

3.1 Block diagram

The block diagram of the Secure Visitor Counter with Crowd Control system illustrates the functional flow of data and control between components responsible for visitor monitoring and crowd management. Each block represents a specific role, ensuring accurate counting, secure access control, and automated operation of the system.

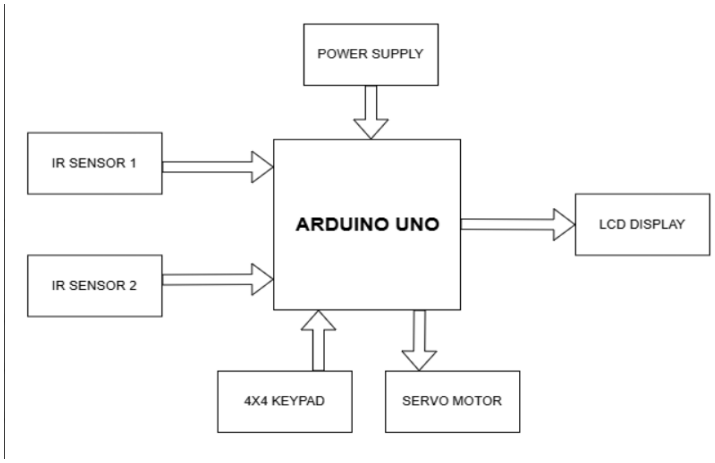


Figure 3.1: Block Diagram

3.2 Circuit Diagram

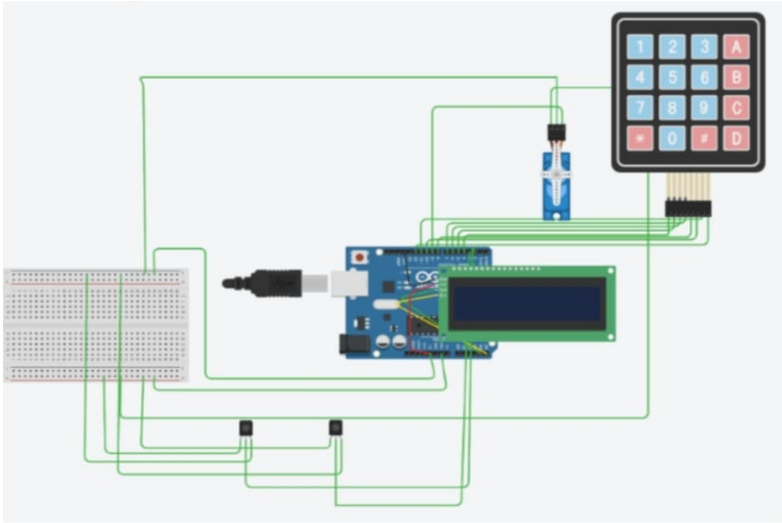


Figure 3.2: Circuit Diagram

Fig 3.2 shows single sensor circuit to detect object and give indication using LEDs. This circuit is made in proteus stimulation software.

Working Process of Secure Visitor Counter with Crowd Control

The working process of the Secure Visitor Counter with Crowd Control system is based on real-time monitoring and logical decision-making using an Arduino Uno. The system accurately tracks the number of people entering and exiting a room and controls access based on a predefined maximum limit.

The step-by-step working process is as follows:

- **Step 1:** System Initialization: When the system is powered on, the Arduino initializes all components. The LCD screen displays the initial visitor count, starting at zero (0). The servo motor remains in the door-closed position by default.

- **Step 2:** Entry Detection and Processing: When a person approaches the entrance, the IR entry sensor detects their presence. The Arduino receives this signal and checks if the current visitor count is below the defined maximum limit (e.g., 10 people). If the condition is met, the system increments the count by one, commands the servo motor to open the door, and then closes it after a brief delay. The updated count is displayed on the LCD screen.

- **step:3** Exit Detection and Processing As a person exits the room, the IR exit sensor is triggered. The Arduino processes this signal by decrementing the visitor count by one. The servo motor opens the door for exit and then returns it to the closed position. The new visitor count is updated and displayed on the LCD.

- **step:4** Crowd Control Enforcement Before allowing a new visitor to enter, the Arduino checks whether the visitor count has reached the predefined maximum limit. If the limit is reached, the door remains closed even when the entry sensor detects a presence. A warning message such as "Limit Reached" is displayed on the LCD. Optionally, a buzzer or LED indicator may activate to alert users or staff. New entries are allowed only after someone exits and the count drops below the maximum limit.

- **Step 5:** Continuous Real-Time Monitoring The system continuously monitors both the entry and exit sensors in real time. The Arduino handles all decision-making and control logic, ensuring the visitor count remains accurate and that the door operates only under valid conditions. This automated process provides efficient, secure, and unattended visitor management.[5]

3.3 List of Components and Budget

Sr.No	Components	Quantity	Price
1	Arduino Uno	01	230 Rs
2	LCD 16x2 Display	01	150 Rs
3	IR Sensor	02	80 Rs
4	Micro Servo Motor	01	40 Rs
5	4x4 Keypad	01	100 Rs

Table 3.1: List of components and their cost

Total Cost = 600 Rs

3.4 Results

Figure 3.3: Model Display

Front view of the Secure Visitor Counter system showing the LCD display, keypad for authorized access, and door mechanism.

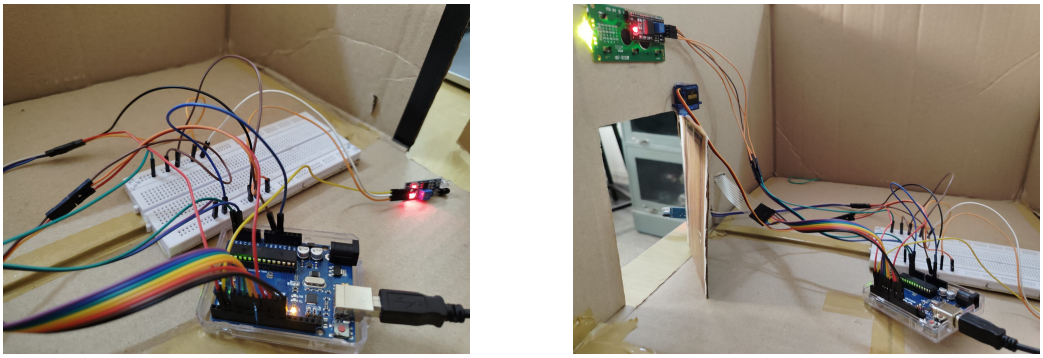
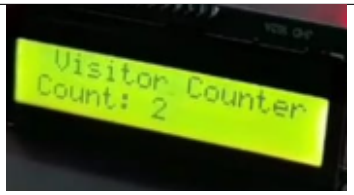
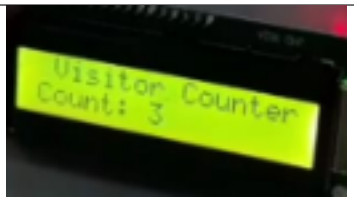
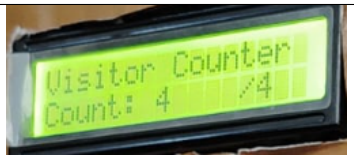


Figure 3.4: Circuit Assembly

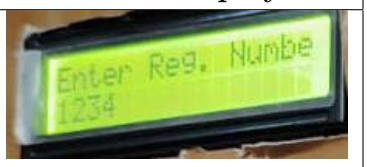
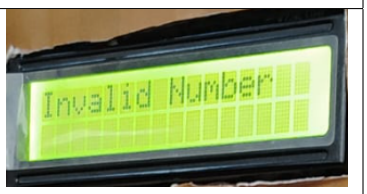
Circuit assembly showing Arduino Uno, IR sensors, servo motor, LCD, and keypad connections.

Table 3.2: Visitor Count Results with Corresponding System Displays

S. No.	Description	Visitor Count Displayed
1	User is asked to enter registration number	
2	No visitors present; initial state of the system	
3	Two visitors detected; system updated accordingly	
4	Three visitors now present inside	
5	Four visitors detected after continuous entry	

3.5 Special Features

Table 3.3: Special Features Operation Displays

S. No.	Description	Feature Displayed
1	Invalid number entered	
2	System gives warning to user	
3	Entry restricted as maximum count reached	

Chapter 4

Conclusion

The "Secure Visitor Counter and Crowd Limiting System" effectively addresses the dual objectives of access control and occupancy management. By utilizing sensor-based entry and exit detection, the system ensures accurate real-time monitoring of the number of individuals within a facility. The integration of a secure access mechanism, which permits entry only upon validation of a registered identification number, enhances overall security. Additionally, the implementation of a maximum occupancy threshold ensures compliance with crowd control protocols by preventing further entry once the limit is reached. This solution offers a reliable and scalable approach for managing access and maintaining safety in environments such as commercial buildings, institutional facilities, and event venues.

Bibliography

[1] Muhammad Ahmad Baballe, Mukhtar Ibrahim Bello, Abubakar Abdullahi Umar, Dahiru Bello, Shehu Atiku Muhammad, and SR Dan Sharif. Visitors counter with gate control and automatic room light controller (on/off). *Global Journal of Research in Engineering & Computer Sciences*, 2(02):32–40, 2022.

[2] Caspar AS Pouw, Joris Willems, and Frank van Schadewijk. Real-time monitoring and crowd control.

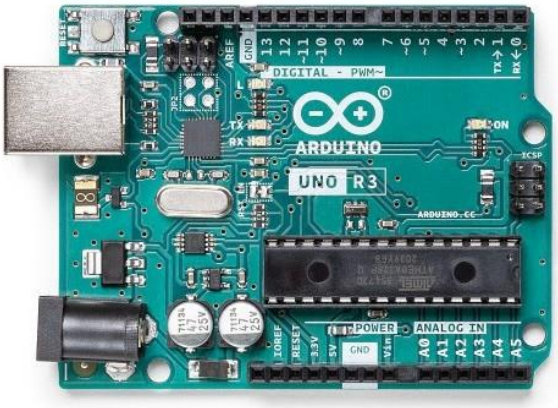
[3] R Sudhakara Pandian, A Madhumathi, Hana Štverková, and P Venkumar. Crowd control and visitor flow in south indian spiritual sites aimed at customer satisfaction. *ZEMCH Network*, page 993.

[4] Divyan Munirathnam Konidala, Robert H Deng, Yingjiu Li, Hoong Chuin Lau, and Stephen E Fienberg. Anonymous authentication of visitors for mobile crowd sensing at amusement parks. In *Information Security Practice and Experience: 9th International Conference, ISPEC 2013, Lanzhou, China, May 12-14, 2013. Proceedings 9*, pages 174–188. Springer, 2013.

[5] Sagir Shehu Muhammad, Musa Dan-azumi Mohammed, and Sadiya Salihu Umar. Gate-controlled bidirectional visitors counter.

Appendix A

Datasheet-1



Description

The Arduino® UNO R3 is the perfect board to get familiar with electronics and coding. This versatile development board is equipped with the well-known ATmega328P and the ATMega 16U2 Processor.

This board will give you a great first experience within the world of Arduino.

Target areas:

Maker, introduction, industries

Features

- **ATMega328P** Processor
  - **Memory**
    - AVR CPU at up to 16 MHz
    - 32 kB Flash
    - 2 kB SRAM
    - 1 kB EEPROM
  - **Security**
    - Power On Reset (POR)
    - Brown Out Detection (BOD)
  - **Peripherals**
    - 2x 8-bit Timer/Counter with a dedicated period register and compare channels
    - 1x 16-bit Timer/Counter with a dedicated period register, input capture and compare channels
    - 1x USART with fractional baud rate generator and start-of-frame detection
    - 1x controller/peripheral Serial Peripheral Interface (SPI)
    - 1x Dual mode controller/peripheral I2C
    - 1x Analog Comparator (AC) with a scalable reference input
    - Watchdog Timer with separate on-chip oscillator
    - Six PWM channels
    - Interrupt and wake-up on pin change
  - **ATMega16U2 Processor**
    - 8-bit AVR® RISC-based microcontroller
  - **Memory**
    - 16 kB ISP Flash
    - 512B EEPROM
    - 512B SRAM
    - debugWIRE interface for on-chip debugging and programming
  - **Power**
    - 2.7-5.5 volts

Board Operation

1.1 Getting Started - IDE

If you want to program your UNO R3 while offline you need to install the Arduino Desktop IDE [1] To connect the UNO R3 to your computer, you'll need a USB-B cable. This also provides power to the board, as indicated by the LED.

1.2 Getting Started - Arduino Cloud Editor

All Arduino boards, including this one, work out-of-the-box on the Arduino Cloud Editor [2], by just installing a simple plugin.

The Arduino Cloud Editor is hosted online, therefore it will always be up-to-date with the latest features and support for all boards. Follow **[3]** to start coding on the browser and upload your sketches onto your board.

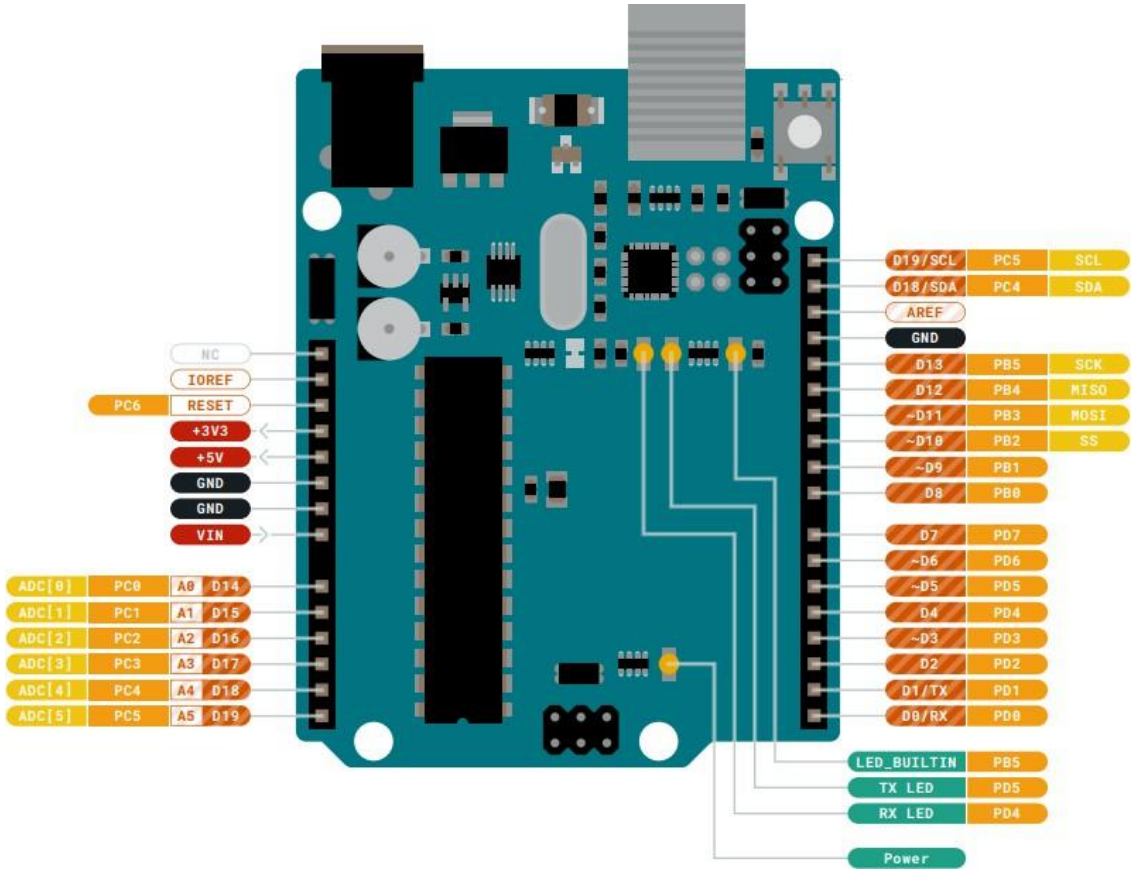
1.3 Sample Sketches

Sample sketches for the UNO R3 can be found either in the "Examples" menu in the Arduino IDE or in the "Documentation" section of the Arduino website [4].

1.4 Online Resources

Now that you have gone through the basics of what you can do with the board you can explore the endless possibilities it provides by checking exciting projects on Arduino Project Hub [5], the Arduino Library Reference [6] and the online Arduino store [7] where you will be able to complement your board with sensors, actuators and more.

2 Connector Pinouts



Pinout

2.1 JANALOG

Pin	Function	Type	Description
1	NC	NC	Not connected
2	IOREF	IOREF	Reference for digital logic V - connected to 5V
3	Reset	Reset	Reset
4	+3V3	Power	+3V3 Power Rail
5	+5V	Power	+5V Power Rail
6	GND	Power	Ground
7	GND	Power	Ground
8	VIN	Power	Voltage Input
9	A0	Analog/GPIO	Analog input 0 /GPIO
10	A1	Analog/GPIO	Analog input 1 /GPIO
11	A2	Analog/GPIO	Analog input 2 /GPIO
12	A3	Analog/GPIO	Analog input 3 /GPIO
13	A4/SDA	Analog input/I2C	Analog input 4/I2C Data line
14	A5/SCL	Analog input/I2C	Analog input 5/I2C Clock line

2.2 JDIGITAL

Pin	Function	Type	Description
1	D0	Digital/GPIO	Digital pin 0/GPIO
2	D1	Digital/GPIO	Digital pin 1/GPIO
3	D2	Digital/GPIO	Digital pin 2/GPIO
4	D3	Digital/GPIO	Digital pin 3/GPIO
5	D4	Digital/GPIO	Digital pin 4/GPIO
6	D5	Digital/GPIO	Digital pin 5/GPIO
7	D6	Digital/GPIO	Digital pin 6/GPIO
8	D7	Digital/GPIO	Digital pin 7/GPIO
9	D8	Digital/GPIO	Digital pin 8/GPIO
10	D9	Digital/GPIO	Digital pin 9/GPIO
11	SS	Digital	SPI Chip Select
12	MOSI	Digital	SPI1 Main Out Secondary In
13	MISO	Digital	SPI Main In Secondary Out
14	SCK	Digital	SPI serial clock output
15	GND	Power	Ground
16	AREF	Digital	Analog reference voltage
17	A4/SD4	Digital	Analog input 4/I2C Data line (duplicated)
18	A5/SD5	Digital	Analog input 5/I2C Clock line (duplicated)

Appendix B

Datasheet-2

InfraRed IR Obstacle Detector

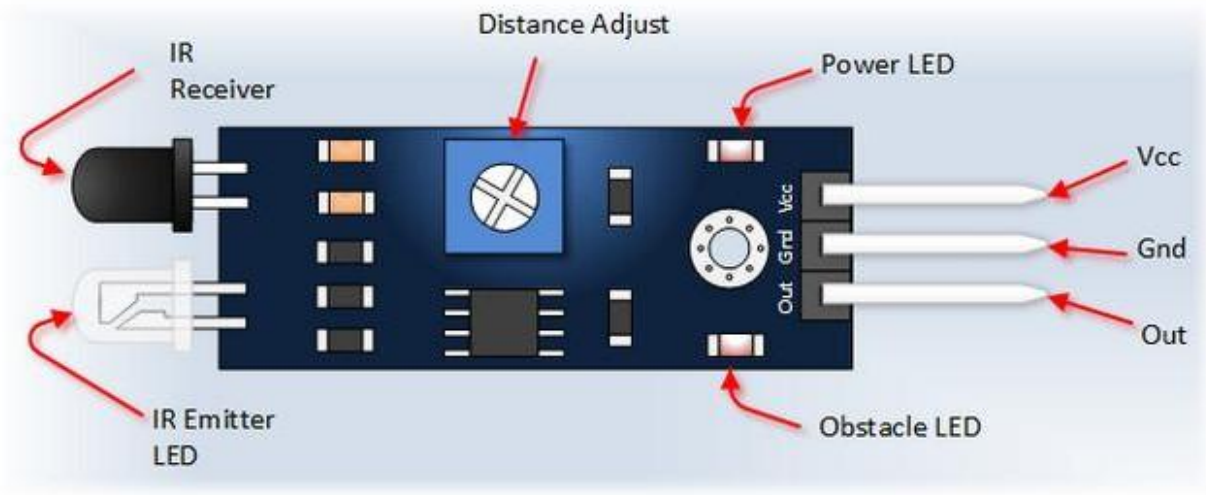
This Infrared Obstacle Sensor has a pair of infrared transmitting and receiving sensors. The infrared LED emits Infrared light and when an obstacle appears on the line of infrared light, it is reflected back by the obstacle which is sensed by the receiver LED.
When the sensor detects an obstacle, the LED indicator lights up, giving a low-level output signal in the OUT pin. The sensor detects distance of 2~30cm. The sensor has a potentiometer which can be adjusted to change the detection distance.

SKU: SSR1017

Brief Data:

- Operating Voltage: 3~5VDC.
- Output type: Digital (0 and 1).
- Detection Distance: 2~30cm. Potentiometer adjustment.
- Mounting Hole: Ø3mm.
- Board size: 3.2 x 1.4cm

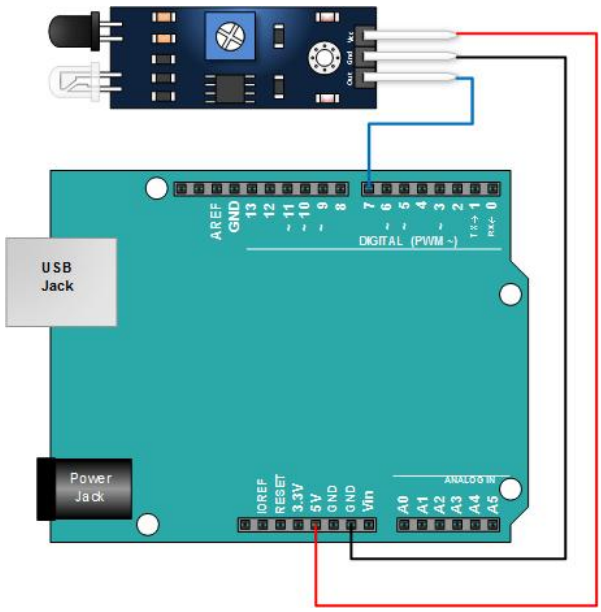
Functional Diagram:



Pin Function	Description
Vcc	3.3 ~ 5Vdc Supply Input.
Gnd	Ground.
Out	Output low when obstacle is in range.
Power LED	Light on when power is applied.
Obstacle LED	Light on when obstacle detected.
Distance Adjust	Adjust detection distance. CCW decreases distance. CW increases distance.
IR Emitter	Infrared emitter LED.
IR Receiver	Infrared receiver that receives signal transmitted by Infrared emitter.

Application with Arduino:

Connect the IR Sensor as below schematic:



Appendix C

Datasheet-3

4x4 Matrix Membrane Keypad (#27899)

This 16-button keypad provides a useful human interface component for microcontroller projects. Convenient adhesive backing provides a simple way to mount the keypad in a variety of applications.

Features

- Ultra-thin design
- Adhesive backing
- Excellent price/performance ratio
- Easy interface to any microcontroller
- Example programs provided for the BASIC Stamp 2 and Propeller P8X32A microcontrollers

Key Specifications

- Maximum Rating: 24 VDC, 30 mA
- Interface: 8-pin access to 4x4 matrix
- Operating temperature: 32 to 122 °F (0 to 50°C)
- Dimensions:
  Keypad, 2.7 x 3.0 in (6.9 x 7.6 cm)
  Cable: 0.78 x 3.5 in (2.0 x 8.8 cm)

Application Ideas

- Security systems
- Menu selection
- Data entry for embedded systems

How it Works

Matrix keypads use a combination of four rows and four columns to provide button states to the host device, typically a microcontroller. Underneath each key is a pushbutton, with one end connected to one row, and the other end connected to one column. These connections are shown in Figure 1.

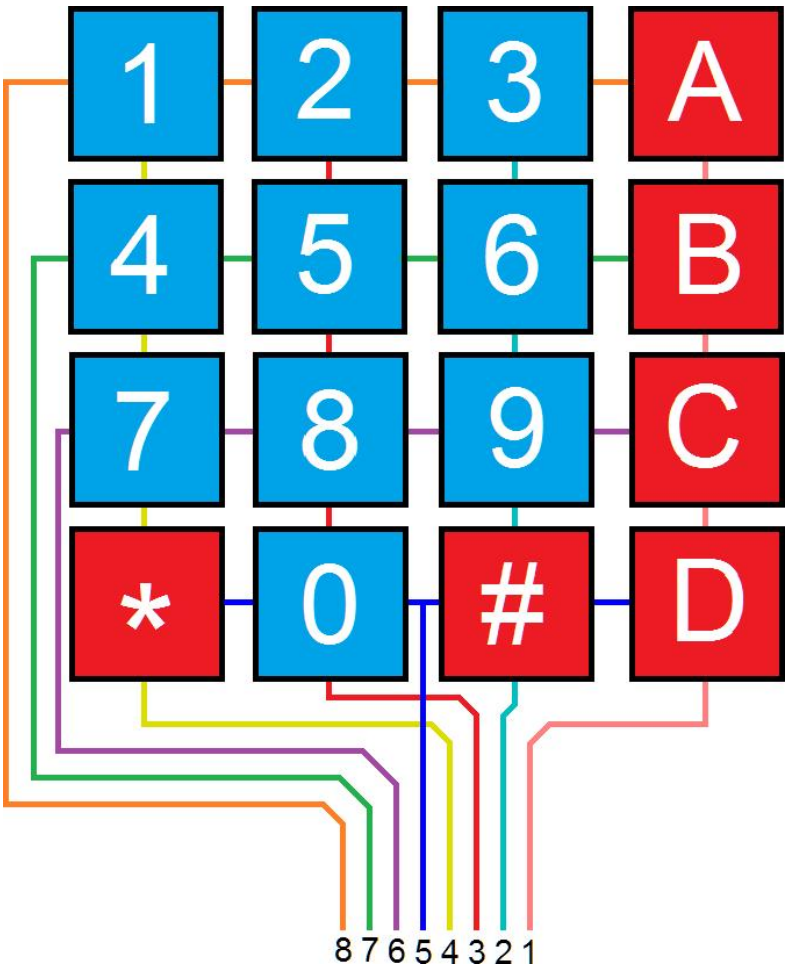
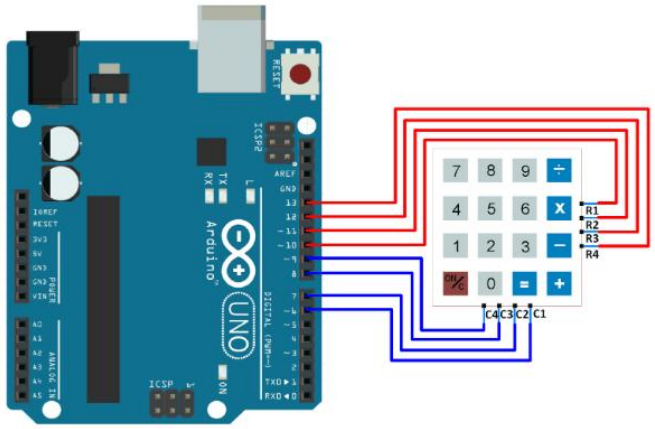


Figure 1: Matrix Keypad Connections

In order for the microcontroller to determine which button is pressed, it first needs to pull each of the four columns (pins 1-4) either low or high one at a time, and then poll the states of the four rows (pins 5-8). Depending on the states of the columns, the microcontroller can tell which button is pressed.

For example, say your program pulls all four columns low and then pulls the first row high. It then reads the input states of each column, and reads pin 1 high. This means that a contact has been made between column 4 and row 1, so button 'A' has been pressed.

Connection Diagrams



Interfacing 4x4 Keypad with Arduino UNO

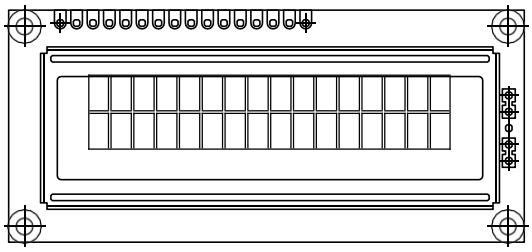
Code

```
#include <Keypad.h>
const byte ROWS = 4; /* four rows */
const byte COLS = 4; /* four columns */ /* define the symbols on the buttons of the keypads */
char hexaKeys[ROWS][COLS] = {
{'0','1','2','3'},
{'4','5','6','7'},
{'8','9','A','B'},
{'C','D','E','F'}
};
byte rowPins[ROWS] = {10, 11, 12, 13}; /* connect to the row pinouts of the keypad */
byte colPins[COLS] = {6, 7, 8, 9}; /* connect to the column pinouts of the keypad */
/* initialize an instance of class NewKeypad */
Keypad customKeypad = Keypad( makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);
void setup(){
Serial.begin(9600);
}
void loop(){
char customKey = customKeypad.getKey();
if (customKey){
Serial.println(customKey);
}
}
```

Appendix D

Datasheet-4

16 x 2 Character LCD



FEATURES

- Type: Character
- Display format: 16 x 2 characters
- Built-in controller: ST 7066 (or equivalent)
- Duty cycle: 1/16
- 5 x 8 dots includes cursor
- + 5 V power supply
- LED can be driven by pin 1, pin 2, or A and K
- N.V. optional for + 3 V power supply
- Optional: Smaller character size (2.95 mm x 4.35 mm)

MECHANICAL DATA

ITEM	STANDARD VALUE	UNIT
Module Dimension	80.0 x 36.0 x 13.2 (max.)	mm
Viewing Area	66.0 x 16.0	
Dot Size	0.55 x 0.65	
Dot Pitch	0.60 x 0.70	
Mounting Hole	75.0 x 31.0	
Character Size	2.95 x 5.55	

ABSOLUTE MAXIMUM RATINGS

ITEM	SYMBOL	STANDARD VALUE			UNIT
		MIN.	TYP.	MAX.	
Power Supply	V_{DD} to V_{SS}	- 0.3	-	13	V
Input Voltage	V_I	V_{SS}	-	V_{DD}	

Note

- V_{SS} = 0 V, V_{DD} = 5.0 V

ELECTRICAL CHARACTERISTICS

ITEM	SYMBOL	CONDITION	STANDARD VALUE			UNIT
			MIN.	TYP.	MAX.	
Input Voltage	V_{DD}	V_{DD} = + 5 V	4.5	5.0	5.5	V
Supply Current	I_{DD}	V_{DD} = + 5 V	1.0	1.2	1.5	mA
Recommended LC Driving Voltage for Normal Temperature Version Module	V_{DD} to V_0	- 20 °C	-	-	5.2	V
		0 °C	-	-	-	
		25 °C	-	3.7	-	
		50 °C	-	-	-	
		70 °C	3.1	-	-	
LED Forward Voltage	V_F	25 °C	-	4.2	4.6	V
LED Forward Current - Array	I_F	25 °C	-	100	-	mA
LED Forward Current - Edge			-	20	40	
EL Power Supply Current	I_{EL}	V_{EL} = 110 V_{AC}, 400 Hz	-	-	5.0	mA

DISPLAY CHARACTER ADDRESS CODE

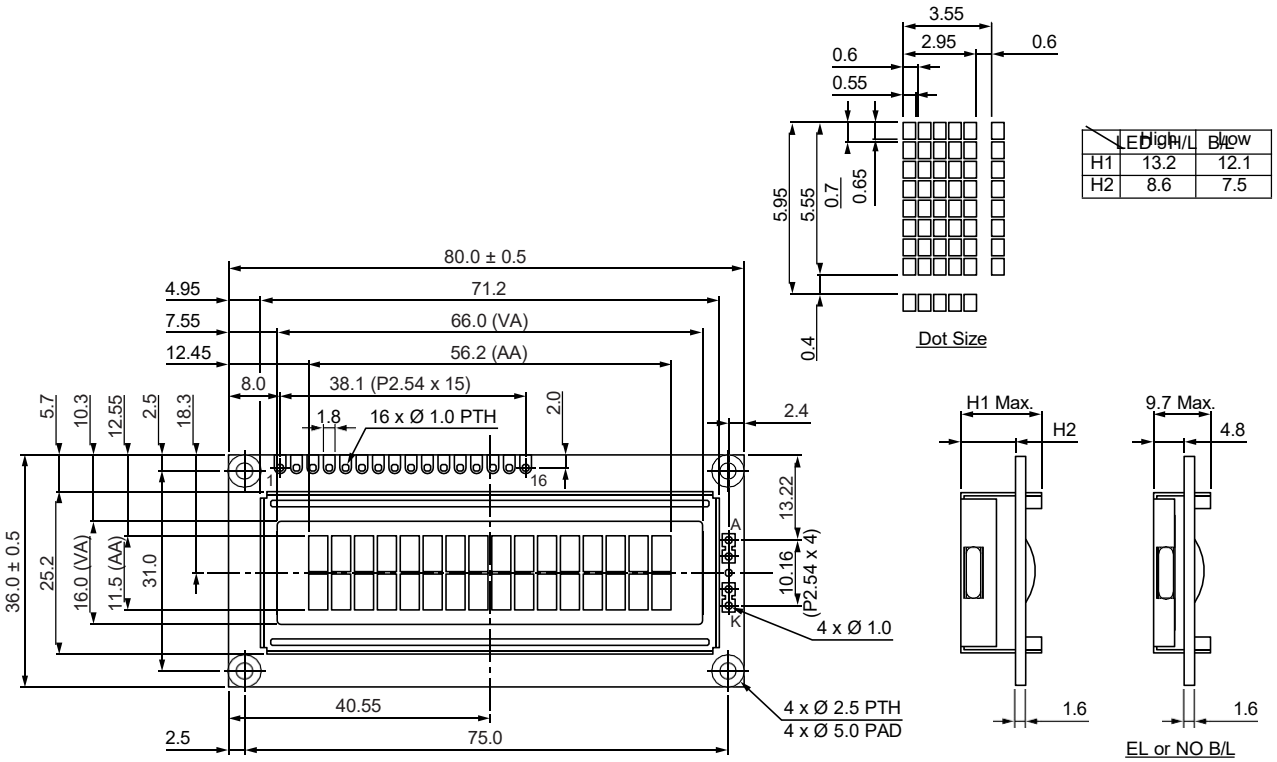
Display Position

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
DD RAM Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
DD RAM Address	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F

INTERFACE PIN FUNCTION

PIN NO.	SYMBOL	FUNCTION
1	V_{SS}	Ground
2	V_{DD}	Supply voltage for logic
3	V_0	Operating voltage for LCD
4	RS	H: Data/L: Instruction code
5	$R/\overline{W}$	H: Read (MPU → Module)/L: Write (MPU → Module)
6	E	H → L chip enable signal
7	DB0	Data bus line
8	DB1	Data bus line
9	DB2	Data bus line
10	DB3	Data bus line
11	DB4	Data bus line
12	DB5	Data bus line
13	DB6	Data bus line
14	DB7	Data bus line
15	A	Supply power for LED+
16	R	Supply power for Red-
17	G	Supply power for Green-
18	B	Supply power for Blue-

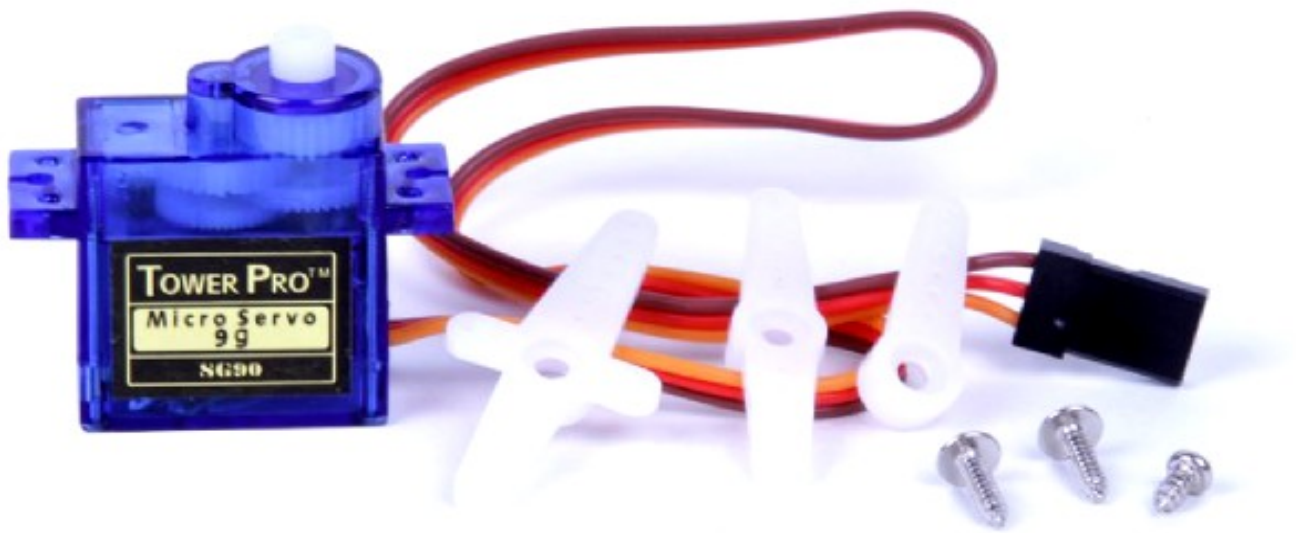
DIMENSIONS in millimeters



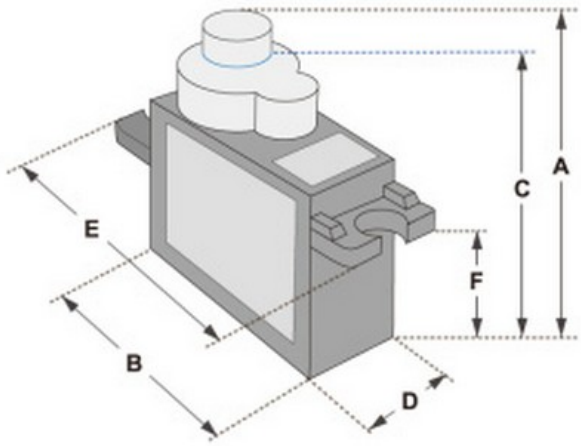
Appendix E

Datasheet-5

SERVO MOTOR SG90 DATA SHEET

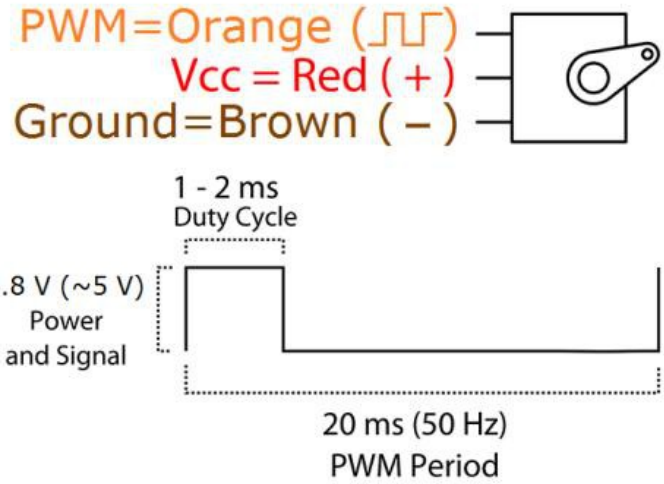


Tiny and lightweight with high output power. Servo can rotate approximately 180 degrees (90 in each direction), and works just like the standard kinds but smaller. You can use any servo code, hardware or library to control these servos. Good for beginners who want to make stuff move without building a motor controller with feedback & gear box, especially since it will fit in small places. It comes with a 3 horns (arms) and hardware.



Dimensions & Specifications
A (mm) : 32
B (mm) : 23
C (mm) : 28.5
D (mm) : 12
E (mm) : 32
F (mm) : 19.5
Speed (sec) : 0.1
Torque (kg-cm) : 2.5
Weight (g) : 14.7
Voltage : 4.8 - 6

Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is middle, is all the way to the right, "-90" (~1ms pulse) is all the way to the left.



Chapter 5

Codes

Arduino Code for Secure Visitor Counter with Crowd Control

```
#include <LiquidCrystal_I2C.h>
#include <Wire.h>
#include <Servo.h>
#include <Keypad.h>

#define ENTRY_SENSOR 2
#define EXIT_SENSOR 3
#define DOOR_SERVO_PIN 9

int visitor_count = 0;
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo doorServo;

bool maxCountReached = false;
bool lastEntryState = HIGH;
bool lastExitState = HIGH;
bool doorMoving = false;
unsigned long lastExitTime = 0;
```

```
const int debounceDelay = 1000;

// Keypad Configuration
const byte ROW_NUM = 4;
const byte COLUMN_NUM = 4;
byte pin_rows[ROW_NUM] = {12, 11, 10, 8};
byte pin_column[COLUMN_NUM] = {4, 5, 6, 7};

char keys[ROW_NUM][COLUMN_NUM] = {
  {'A','3','2','1'},
  {'B','6','5','4'},
  {'C','9','8','7'},
  {'D','#','0','*'}
};

Keypad keypad = Keypad(makeKeymap(keys), pin_rows,
pin_column, ROW_NUM, COLUMN_NUM);

String validRegistrationNumbers[] = {"1238", "5678", "9101", "1121"};
const int maxVisitors = 4;
String enteredNumber = "";

void updateDisplay();
bool isValidRegistrationNumber(String number);
String getRegistrationNumber();
void handleEntry();
void handleExit();

void setup() {
    pinMode(ENTRY_SENSOR, INPUT);
    pinMode(EXIT_SENSOR, INPUT);

    Serial.begin(9600);
```

```
    lcd.init();
    lcd.backlight();
    lcd.clear();

    doorServo.attach(DOOR_SERVO_PIN);
    delay(100);
    doorServo.write(0);

    updateDisplay();
}

void loop() {
    if (!doorMoving) {
        bool entryState = digitalRead(ENTRY_SENSOR);
        bool exitState = digitalRead(EXIT_SENSOR);

        // Handle Entry (Debounced)
        if (entryState == LOW && lastEntryState == HIGH) {
            handleEntry();
        }
        lastEntryState = entryState;

        // Handle Exit (Debounced)
        if (exitState == LOW &&
        lastExitState == HIGH
        && (millis() - lastExitTime > debounceDelay)) {
            handleExit();
            lastExitTime = millis();
        }
        lastExitState = exitState;
    }
}
```

```
void handleEntry() {
    if (visitor_count < maxVisitors) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Enter Reg. Number");
        enteredNumber = getRegistrationNumber();

        if (isValidRegistrationNumber(enteredNumber)) {
            visitor_count++;
            Serial.print("Visitor Entered | Count: ");
            Serial.println(visitor_count);
            updateDisplay();
          \\Motor Operation
            doorMoving = true;
            doorServo.write(90);
            delay(3000);
            doorServo.write(0);
            delay(1000);
            doorMoving = false;
        } else {
            lcd.clear();
            lcd.setCursor(0, 0);
            lcd.print("Invalid Number");
            delay(2000);
            lcd.clear();
        }
    } else {
        maxCountReached = true;
        updateDisplay();
    }
}
```

```
// Visitor Exit
void handleExit() {
    if (visitor_count > 0) {
        visitor_count--;
        Serial.print("Visitor Exited | Count: ");
        Serial.println(visitor_count);
        maxCountReached = false;
        updateDisplay();
    }
}

// Registration Number from Keypad
String getRegistrationNumber() {
    String number = "";
    char key;
    lcd.setCursor(0, 1);

    while (true) {
        key = keypad.getKey();
        if (key) {
            if (key == '#') {
                if (number.length() == 4) {
                    return number;
                } else {
                    lcd.setCursor(0, 1);
                    lcd.print("Enter 4 digits! ");
                    delay(2000);
                    lcd.setCursor(0, 1);
                    lcd.print("                ");
                    number = "";
                    lcd.setCursor(0, 1);
                }
            } else if (key == '*') {
```

```
                number = "";
                lcd.setCursor(0, 1);
                lcd.print("----------------");
                lcd.setCursor(0, 1);
            } else if (number.length() < 4) {
                number += key;
                lcd.print(key);
            }
        }
    }
}


bool isValidRegistrationNumber(String number) {
    int listSize = sizeof(validRegistrationNumbers)
    sizeof(validRegistrationNumbers[0]);
    for (int i = 0; i < listSize; i++) {
        if (validRegistrationNumbers[i] == number) {
            return true;
        }
    }
    return false;
}

void updateDisplay() {
    lcd.clear();
    if (maxCountReached) {
        lcd.setCursor(0, 0);
        lcd.print("Max Count Reached");
        lcd.setCursor(0, 1);
        lcd.print("Entry Prohibited");
    } else {
        lcd.setCursor(0, 0);
```

```
        lcd.print("Visitor Counter");
        lcd.setCursor(0, 1);
        lcd.print("Count: ");
        lcd.print(visitor_count);
        lcd.setCursor(12, 1);
        lcd.print("/4");
    }
}
```